User Modeling and User-Adapted Interaction manuscript No.
(will be inserted by the editor)

A Model for Profile Management Through Inference in Trails

André Wagner · Jorge Luis Victória Barbosa · Débora Nice Ferrari Barbosa

Received: date / Accepted: date

This paper or a similar version is not currently under review by a journal or conference, nor will be submitted to such within the next three months. This paper is void of plagiarism or self-plagiarism as defined in Springer's Policy on Publishing Integrity.

Abstract Context-aware and ubiquitous systems have the challenge of implicitly collect relevant information about entities, and use this information to understand and predict their behaviour. This allows the applications to adapt themselves to the the entities, thus avoiding to overflow it with inquires and information. This article presents a model that allows applications to register users' actions in trails and infer profile information from these trails, using semantic interoperability and thus allowing different applications to share information and infer a unified profile. The contributions of this model are the use of trails for extracting profiles, the generation of dynamic profiles and the capability of managing dynamic inference rules for profile generation.

Keywords Profile Management · Ubiquitous Computing · Trails · Profile Inference

André Wagner · Jorge Luis Victória Barbosa
Universidade do Vale do Rio dos Sinos (UNISINOS)
Unidade Acadêmica de Pesquisa e Pós-Graduação
Av. Unisinos, 950, São Leopoldo, Brazil
E-mail: andre.nho@gmail.com, jbarbosa@unisinos.br

Débora Nice Ferrari Barbosa
Universidade Feevale
Gr. Pesquisa em Informática na Educação
RS-239, 2755, Novo Hamburgo, Brazil
E-mail: deboranice@feevale.br

1 Introduction

Nowadays, most users interact with a wide variety of computing systems. The current tendency is the increased interconnectedness between these systems, since the internet is becoming every day more present and available. Recent research predicts the full integration between these systems in a federation of computing systems. This federation is referred to as Smart Environment (Heckmann, 2005).

Weiser (1991) discussed a vision for the concept of smart environments. He presented a scenario in which the computer would become invisible and indistinguishable from its environment. In this scenario, interaction between humans and computers would not occur as usual – where the humans devote their full attention to the computer –, but would consist of small and quick interactions, usually in different devices and locations (Weiser & Brown, 1996). According to Weiser (1991), "the most profound technologies are those that disappear. They weave themselves into the fabric of everyday life until they are indistinguishable from it." He asserts that the future of computing lies in integrating the computer to the real world, not the opposite, and he calls this new paradigm Ubiquitous Computing.

However, the availability of computing resources at any time and everywhere poses the risk of information overload on the user. To solve this problem, the concept of "calm technology" was created (Weiser & Brown, 1996). This means the transfer of information between the computer and the users must happen in the "periphery" of the users' attention, moving to the "center" of their attention in moments of urgency. A car's engine can be taken as an example. When a person drives a car, the attention is focused on the road, and the driver almost doesn't notice the engine sound. However, if the engine emits a strange sound (that might indicate some kind of problem), the driver's attention turns to the engine.

If a system is to be minimally intrusive, it must understand the situation of the user and his surroundings, and adapt its own behaviour according to this information. This information is called Context, and systems capable of this kind of adaptation are called context-aware (Satyanarayanan, 2001; Dey, 2001).

Research in this area shows that is not only useful to know the context of the user when a service is being offered, but that the history of visited contexts can be a valuable information to help the application to adapt itself to the user. This history of visited contexts is called Trail (Silva et al., 2009). Although the trail is usually tied to the location of a user, the concept of a trail is broader and includes activities performed, applications used, content accessed – among others – inside a specific context and timeframe (Silva et al., 2010).

For an application to adapt itself effectively to a user, it is not only important to know the history of contexts visited by a user, but to understand who the user is – that is, what are his characteristics. This kind of application

organize its knowledge about a user in User Profiles. The profiles are stored in a format called User Model (Viviani et al., 2010).

Personalization means the individual adaptation of products, services and information. Personalization technology usually involves programs that learn patterns, habits and preferences of a user. The main goal of modern personalization systems is to offer the users what they want, without demanding them to inform it explicitly (Mulvenna et al., 2000). It is clear that this purpose meets the ubiquitous computing objective that is, the development of a technology that "disappears".

A relevant aspect for environments with multiple independent systems is the interoperability. Interoperability is the ability of two or more systems or components to exchange information and use the information that has been exchanged. (IEEE, 1991). In the context of customization, interoperability refers to the ability to access and interpret information derived from multiple heterogeneous sources, and integrate it into a common user model. Interoperability can be (i) syntactic, where the differences between the models are resolved at application level, or (ii) semantic, where the differences between the models are resolved at the level of knowledge (Carmagnola & Cena, 2009).

Research indicates that it is desirable that future systems become increasingly invisible to their users (Weiser & Brown, 1996), and that a system can only become invisible if it is proactive, and can only be proactive by knowing the user (Satyanarayanan, 2001). For a system to know a user, it must know the user history, understand the context in which he is inserted, as well as data collected concerning him in the same context (or in a similar context), and be able to process this set of information into a user profile.

In this paper, the concept of entity is used instead of the classic concept of User. While usually the "user" means the person who is using the application, an entity refers to any object (physical or not) able to control an application. Thus, an entity can be a person, a moving object (like a car) or property (like a room), or even a computer system.

In this setting we are proposing eProfile, a model for the automatic creation of profiles by inference of entity trails.

The main contributions of this model are:

1. The use of trails for extracting profiles. By looking at the entity history, it is possible not only to consider its current situation, but also to examine the entity's past, obtaining a more complete set of information. Furthermore, this allows the entity profile to be generated automatically, instead of requiring the entity or application to fill out the data manually;
2. The capability of generating dynamic profiles. The continuous creation of trail events allows the profile to be constantly updated, thus allowing the automatic adaptation of the profile to the entities' actions.
3. The capability of managing dynamic inference rules and models. The applications can change the inference rules and entity models at any moment, which give the developers a opportunity to build applications that can adapt themselves to their users;

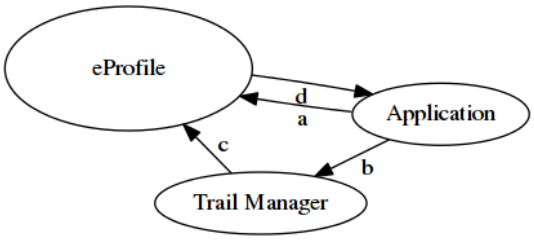

Fig. 1 Overview of eProfile

4. The semantic operability of the model. The ability to use information generated by various applications to build an entity profile allows the application to create more comprehensive profile generation rules, thus enriching the profiles information.

This article is organized as follows. Section 2 proposes the eProfile model. Section 3 discusses implementation aspects. Section 4 presents an experiment validating the model. Section 5 describes related work, showing the contributions of this model. Finally, a Section 6 presents the concluding remarks.

2 eProfile Model

This section describes the eProfile model. Section 2.1 presents a general vision of the model. Section 2.2 explains the model's architecture. Section 2.3 demonstrates the ontologies used by the model, and Section 2.4 explains how the profiles are inferred.

2.1 General Vision

Figure 1 shows the overview of eProfile, where each circle represents a component, and the arrows indicate the flow of data.

The **application** is any piece of software that is being used directly by an entity. For an application to be considered suitable to use this model and the functionality offered by eProfile, it needs to record its information on a trail manager supported by eProfile. The application must also implement a communication protocol to exchange messages with eProfile. If these prerequisites are followed, the application can be of any type and operate in any type of device that has communication with the Internet.

The second component depicted is a **trail manager**. A trail manager is capable of storing events held by an entity, putting them in chronological order and registering information relevant to the event (eg, the context in which the event was held). eProfile supports any trail manager whose communication protocol is implemented in eProfile.

The third component of Figure 1 represents the **eProfile**. The arrows indicate the flow of data, and the letters used in this figure represent the same

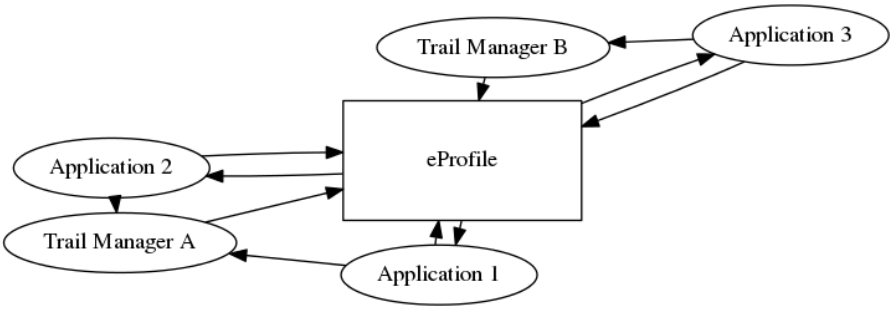

Fig. 2 eProfile connected to multiple trail managers

ones used in Figure 3. The communication starts when the application registers itself on eProfile (arrow "a"), sending (i) the entity model used by the application and (ii) the rules for construction of this model. From this moment on, the application starts recording the events on the trail manager (arrow "b"). The eProfile periodically retrieves this information from the trail manager and builds or reconfigures the profile entity (arrow "c"). When the application needs an updated profile, it retrieves this profile from eProfile (arrow "d").

Melhorar o texto justificando o uso de agentes

Since eProfile supports multiple applications, it is possible for different applications to use different trail managers. Therefore, eProfile also supports communication with a number of trail managers, as exemplified in Figure 2.

2.2 Architecture

Figure 3 shows the architecture of eProfile. The components "Application" and "trail manager" are the same as shown in Figure 1, while the "eProfile" service was expanded to show its inner workings.

Since eProfile is a model that infers information about data stored in an ontology autonomously, and is organized into independent parts, it uses the technology of autonomous agents.

Each application builds its own entity model. The eProfile needs three pieces of information to be able to process the profiles: (i) the entity trail, (ii) the entity model used by the application and (iii) the inference rules for the profile. This can be seen in Figure 3, where the agent "Reasoner" receives data from these three databases. The trail of the entity is loaded by eProfile from the trail manager, while the other two information are sent directly from the application to eProfile. Thus, the inference rules defined by the application are used to infer the profile of the entity.

The arrows in Figure 3 indicate the direction of the data flow, where it can be noted two entry points and one exit point.

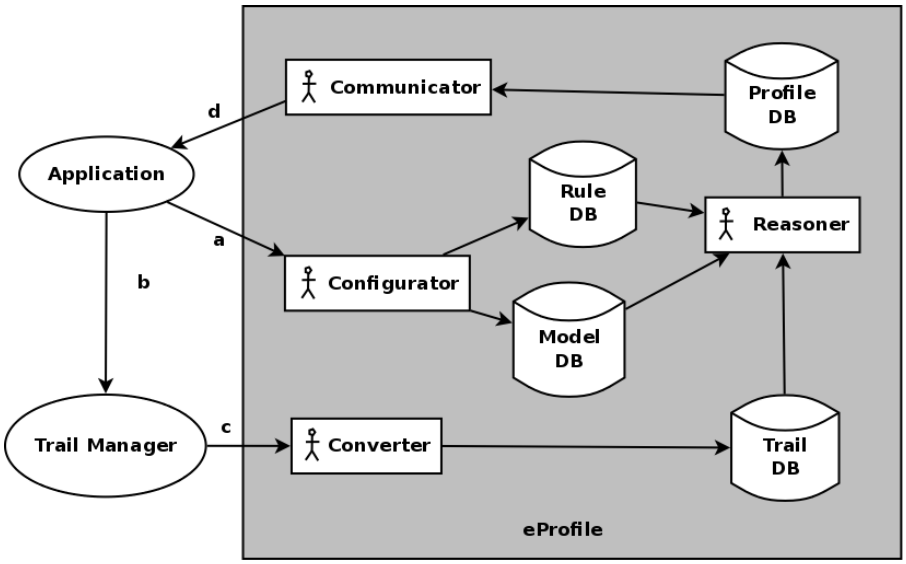

Fig. 3 eProfile architecture

The first entry point (arrow "a") is the information sent from the application to eProfile or, more specifically, to the **Configurator** agent. The following information must be sent by the application:

- The entity model used by the application, which is the format of the entity profile. Once the entity trail is processed, the inferred data is stored and made available for application using this format;
- The rules used to infer the profile. These rules will be executed at regular intervals in order to update the entity profile. The Configurator receives the rules, validate them and insert them in **rule database**. Along with the rules, is specified the criticality of information. The model uses this information to determine how often the information will be inferred.

The second entry point (arrow "c") allows receiving the entity trails, which are fetched by eProfile from the trail manager. Since eProfile can connect to different trail managers, (see Figure 2), and each trail manager might use one specific format, a conversion between the original format and the format used by eProfile is needed for the trails to be stored in a standardized manner. This work is performed by an agent named **Converter**, which is able to perform the conversion to the eProfile internal trail format.

The entity trail is then stored in a trail ontology called **eOntoTrail**. This ontology contains the basic classes supporting the construction of a trail, and they can be extended according to the specification of the trail manager that is being used.

Once in possession of the trails, the model and rules, the **Reasoner** agent infers from the trail, generating a entity profile that consists of an extension of

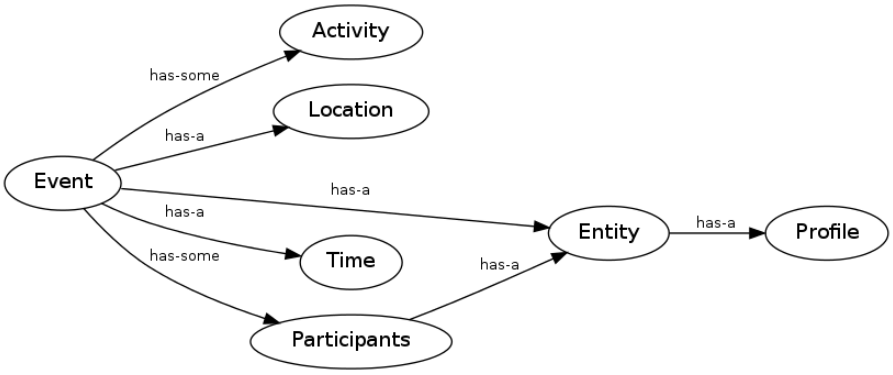

Fig. 4 eProfile Trail Ontology (eOntoTrail)

a basic profile ontology named **eOntoProfile**. This ontology, just like the trail ontology, contains basic classes from which applications can specialize, using them to store their own information. The application can then connect to eProfile and get its entity profile, whose data are sent by the **Communicator** agent (arrow "d").

2.3 Ontologies

As mentioned in Section 2.2, eProfile uses two ontologies: a trail ontology and a profile ontology. These are both basic ontologies, extended by subclasses according to the trail format sent by the trail manager (for the trail ontology) or the needs of the application (for the profile ontology).

The trail ontology data (eOntoTrail), shown in Figure 4, is fed by different supported trail managers. The Converter agent is responsible for receiving data in the trail manager format and converting it to the eOntoTrail. This way, the reasoner has a single ontology on which to reason.

Context-aware applications are interested in knowing **who**, **where**, **when** and **what** (that is, what the user is doing), so they can discover why the situation is occurring (Abowd et al., 1999). Thus, a trail is represented by a set of events characterized by the following main classes: (i) *Entity*, the identity of the entity that is performing the action, which can optionally be linked to a profile, (ii) *Location*, the place where the action is taking place, (iii) *Time*, the date and time the event occurred, (iv) *Participants*, the entities linked to this action, and (v) *Activity*, the action that the entity is performing (or trying to perform). This lastclass is the most comprehensive of the ontology, and will possibly be the most specialized by the trail managers.

The profile ontology (eOntoProfile), shown in Figure 5, is the ontology used in the profile information inferred from the trail, according to the entity model defined by the application. For the ontology to be feasible within the proposed model, it must be a generic ontology that allows its classes to be extended

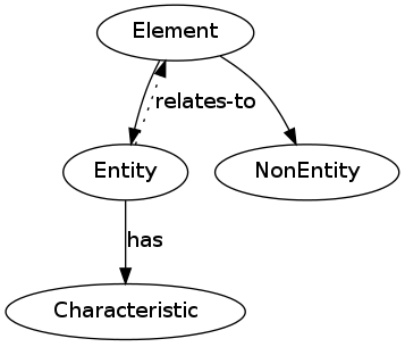

Fig. 5 eProfile Profile Ontology (eOntoProfile)

by applications. In the figure, a relationship between classes is shown by a dotted line, and a specialization of a class is shown by a continuous line. The eOntoProfile is a basic ontology, extended by the applications for them to assemble their own entity models.

The basic class of the ontology is the *Element* class. This class *Element* represents anything in the real world. It is extended by two classes: (i) *Entity*, which represents an entity, that is, an element that has a profile associated with it, and (ii) *NonEntity*, which represents an element than doesn't have a profile associated with it. For example, a trail registers a event in which a person enters a store, but the store has no profile registered in eProfile. In this case, the person is a *Entity* but the store is a *NonEntity*. This distinction is important because in the first case, eProfile has access to a wide range of information about the entity, entities related to that first entity, and so on, while in the second case the store is just an object that eProfile doesn't know nothing about it except for the name. The *Entity* in this ontology is the same as the *Entity* in the eOntoTrail ontology (see Figure **??**).

The class *Characteristic* represents an aspect of the entity, and is the class that will be extended by applications to refer to any information related to that entity. These characteristics can be simple labels (such as name of a person, or mileage of a car) or more complex information (such as academic knowledge or latest routes traveled).

2.4 Rule Execution

The execution of rules is key to eProfile's operation. An inference rule is an instruction issued by an application, that runs on an entity trail, resulting in an entity profile according to an entity model previously registered by the application.

For a rule to be executed, the application must have previously registered within eProfile: (i) the rule itself, (ii) an entity model, which serves as basis for the entity profile format, and (iii) the trail of the entity on which the inference

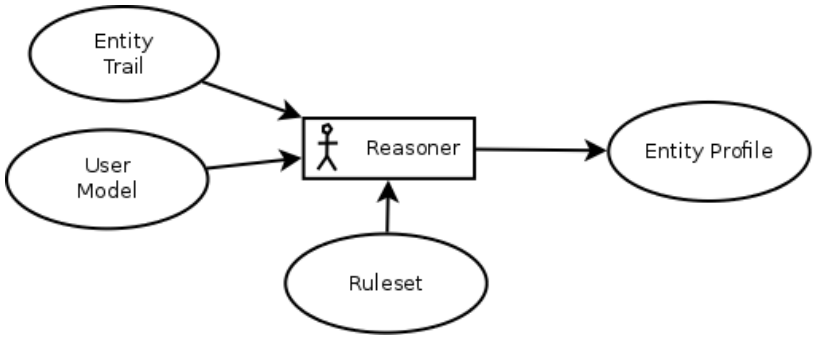

Fig. 6 Data flow for the entity profile inference

is performed. From these three pieces of information, the reasoner is able to generate a profile entity, according to Figure 6.

The rules are always defined by the application, and a rule is composed of the following set of information: (i) rule name, (ii) profile field that will be updated, (iii) the rule, written in OWL language (W3C, 2011), (iv) update method (replacement or update), (v) update frequency, and (vi) expiration of the rule.

The update method defines how the value will be changed in every execution of the rule. There are two options: (i) replacement, where the value will be replaced by a new information (for example, in the case of a person who changes its address); and (ii) update, where the value is updated according with a formula, that can take into consideration the current value (before the update). The update option could be used, for example, in a field that stores the interest of a person in a particular subject. The interest doesn't change radically, but is changed smoothly based in the previously registered interest.

The rules and the entity model can be changed at any moment during the execution of eProfile. A change in the rules or in the model will reflect in a change in the generated profile when the rules are executed the next time.

3 Implementation Aspects

eProfile prototype was developed in Python (Rossum, 1995). Python is a generic, open source, object-oriented, high-level language, that supports multiple architectures. The rule, model and profile databases are implemented using the SQLite3 database (Hwaci, 2012). SQLite3 is an SQL transactional self-contained database, available in the public domain. The SQLite3 was chosen due to its good performance and the fact that it does not require a separate server for deployment. The use of a relational database to store the rule, model and profile databases was made due to performance reasons.

Figure 6 shows the data flow for the entity profile inference process. At the center of this operation is the Reasoner agent, which uses an external inference engine to implement the rules. The reasoner used is FACT++, a

propositional logic reasoner created as a platform for use of algorithms of semantic *tableaux* and optimization techniques (Tsarkov & Horrocks, 2006). This reasoner was chosen due to its good performance and to support all functions used by eProfile.

The knowledge representation language used for storage and processing of ontologies is OWL(W3C, 2011), chosen because for being the most expressive ontology language defined for the Semantic Web (Daconta et al., 2003).

Communication between these three applications (both simulated applications and eProfile) is done through REST web services (Fielding, 2000).

4 Evaluation Aspects

It is only possible to evaluate the functionalities of a proposed infrastructure by building applications that utilize it and then evaluate them, in order to obtain an indirect evaluation of the model (Edwards et al., 2003). The scientific community has been using scenarios to perform this type of indirect evaluation in context sensitive (such as the approach taken by Dey et al. (2001)) and ubiquitous (according to Satyanarayanan (2001)) environments. This work follows the same approach – we created a prototype and a scenario for performing an experiment, in order to evaluate eProfile.

The experiment presented is based on the generation of simulated trails of undergraduate students within the context of Ubiquitous Learning. Ubiquitous Learning can be understood as a set of educational processes that occur anywhere, at any time and with any device, continuously, contextualized and integrated into the daily lives of apprentices (Barbosa et al., 2011).

The following sections describe the simulation environment (Section 4.1), the applications used (Section 4.2), the eProfile/LOCAL integration (Section 4.3) and eProfile/UniManager integration (Section 4.4), trails, rules and generated profiles (Section 4.5) and the simulation parameters and results (Section 4.6).

4.1 Environment

The scenario consists of a group of students that carries out a graduation course in Computer Science. The course consists of a total of 40 disciplines, of which 12 are optional (the student must complete at least 8 out of these 12). All disciplines have other disciplines as prerequisites, except for the disciplines of the beginning of the course (Figure 7 exemplifies a subset of subjects with their prerequisites). Each discipline has a professor, a limited number of openings for students, and a set of necessary resources (such as special rooms or equipment).

The full set of disciplines used for this scenario is shown in Table 1, where the ones marked with an asterisk are optional. Some require a lab (with simple computers), a network lab or hardware lab. The courses without mandatory resources only require a classroom.

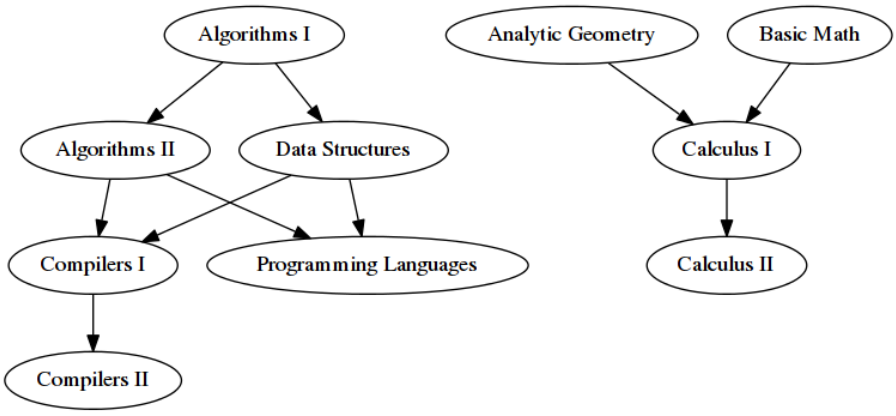

Fig. 7 Prerequisites of some disciplines

Students and professors use mobile devices (laptops or smartphones), on which the applications used for administration of their academic life are installed – in particular, UniManager and LOCAL.

UniManager is the system used for discipline administration, and it is described in detail in Section 4.4. It is used by the university administration, the professors and the students. The administration uses it to register disciplines, students and professors, to link students and professors to their disciplines and to issue management reports such as attendance and approval/failing of students, as well as absences and evaluations of professors. Professors use the system to conduct student evaluations. Students use it to hand out their tests and papers, to make their enrollment in the disciplines and to check their grades and absences.

In addition, professors and students also use LOCAL on their mobile devices. LOCAL is described in grater detail in Section 4.3 and in (Barbosa et al., 2011). LOCAL links information from profiles to their location to be able to assist them, facilitating interaction between learners and enabling the delivery of contextualized educational material. Professors use the LOCAL to register material that will be distributed to students. LOCAL communicates with students to distribute educational material previously registered and to put students with similar interests in touch during educational activities. If, during these activities, the student has not registered any interest that matches those required by the activity, he has the option to make the choice using a user interface to LOCAL called Personal Assistant.

Table 2 exemplifies a two and a half hours class involving 10 students, in which there was a debate about Programming Techniques. After the debate, students undertook a study about the individual topics discussed in the debate. In this example, the presence of students, the handing out of their work and the grade achieved were registered in UniManager. The educational content of the lesson was registered in LOCAL.

Code	Name	Prerequisites	Openings	Resources
A1	Algorithms I	None	60	Lab
A2	Basic Math	None	60	–
A3	Analytic Geometry	None	40	–
A4	Portuguese	None	80	–
A5	System analysis I	None	40	–
B1	Algorithms II	A1	40	Lab
B2	Data Structures	A1	30	Lab
B3	Calculus I	A2, A3	40	–
B4	System Analysis II	A5	40	–
B5	English	None	50	–
C1	Compilers I	B1, B2	25	Lab
C2	Programming Languages	B1, B2	40	Lab
C3	Computer Architecture	None	10	HW Lab
C4	Software Engineering	B4	30	–
C5	Philosophy	A4	30	–
D1	Compilers II*	C1	20	Lab
D2	Microprocessors*	C3	20	HW Lab
D3	Calculus II*	B3	30	–
D4	Databases	A5	30	Lab
D5	Networking*	None	20	Network Lab
E1	Thesis I	C1, C2, C3, C4, D4	15	–
E2	Digital Electronics*	D2	10	HW Lab
E3	Artificial Intelligence*	C4, D3	20	–
E4	Graphic Computing*	C4, D3	20	Lab
E5	IT Management*	C4, D4	20	–
F1	Thesis II	E1	15	–
F2	Robotics*	E2	10	HW Lab
F3	Operating System*	D1, D2, C4	25	–
F4	Project Management*	E5	15	–
F5	Advanced Algorithms*	B1, B2	15	Lab

Table 1 Disciplines used in this scenario

In this example, LOCAL registered the personal preferences of two students about programming techniques. If LOCAL and UniManager were being used in isolation, this information would have been relevant only to the immediate lesson assignment. However, the integration of these systems with eProfile allows this information to be used to update the profile of the students, and can be used in the future to facilitate interaction between students and personalize the delivery of educational content (by LOCAL). Furthermore, a single profile can be generated by the trails from both applications, this information can be used to assist the student in the recommendation of optional disciplines (by UniManager).

Event	Time	Actor	Action
1	-	Professor	Registers educational material that will be used in class into LOCAL, and requests the handing out of a paper through UniManager.
2	7:30	Students	Nine students enter the classroom.
3	7:30	LOCAL	LOCAL identifies the entrance of the students into the classroom.
4	7:30	UniManager	UniManager identifies the entrance of the students into the classroom.
5	7:30	LOCAL	LOCAL sends the class educational material to the students.
6	8:00	Professor	The professor requests the formation of groups for the debate.
7	8:00	LOCAL	LOCAL forms three groups based on the students' preferences: *Distributed Algorithms* (3 students), *Scheduling Algorithms* (3 students) and *Math Algorithms* (1 student). Two students have no registered preference that would allow LOCAL to put them in some group.
8	8:00	LOCAL	LOCAL request the two students to choose a group.
9	8:10	Students	Students indicate their preference through LOCAL Personal Assistant.
10	8:10	LOCAL	LOCAL puts the two students in their chosen groups, one of them in *Scheduling Algorithms* and the other one in *Math Algorithms*.
11	8:10	Students	The debate starts.
12	8:40	Students	One student enters the classroom, late. UniManager and LOCAL register their entrance, but LOCAL doesn't integrate him in the debate, since it's almost over.
13	8:50	Students	The debate ends.
14	8:50	Professor	The professor request the students to write a paper about the debate subject.
15	10:00	Students	The students hand out their papers through UniManager and leave the classroom.
16	10:00	UniManager	Receives the papers and registers the students exit.
17	10:00	LOCAL	Registers the students exit.
18	-	Professor	The professor, later, evaluates the students papers and registers their grade on UniManager.

Table 2 Example of a class

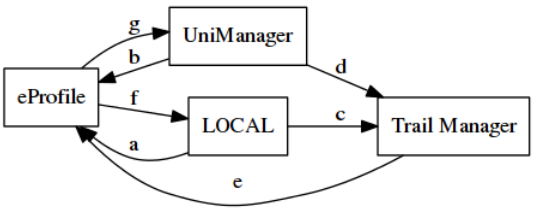

Fig. 8 Scenario model

4.2 Applications

The scenario is composed of four applications that interact among themselves, according to Figure 8. LOCAL is a ubiquitous learning system used to find learning opportunities by putting students in touch and recommending learning content (Barbosa et al., 2011). UniManager is an application used by the administration of the university to manage student grades and disciplines. Both softwares have profile modules, and the profile information is used for making decisions. Since in both cases the profile module is static and depends on collection of explicit information, eProfile is connected to these softwares, leveraging the dynamic creation of profiles and collecting information implicitly. Furthermore, eProfile enables interoperability between these systems by allowing information from both systems to be used to generate single profiles. The scenario also includes a trail manager, that manages the entities trails. In this scenario, the disciplines, students and professors are represented as entities.

The initial communication occurs between applications and eProfile (arrows "a" and "b"), where applications define their entity models and rules. This data exchange can occur as often as necessary, because applications can update their rules and their entity models at any time. During the execution of the applications, they record the events in the trail manager (arrows "c" and "d"). This means that applications can run disconnected from eProfile, and connect only when a rule needs to be updated or when profile data must be downloaded. eProfile retrieves trails periodically from the trail manager (arrow "e"). When eProfile is in possession of the trails, rules and entity models, it starts to process this information, inferring new profile data. These profiles are then downloaded by the applications (arrows "f" and "g").

4.3 eProfile/LOCAL Integration

This scenario integrates eProfile to a ubiquitous learning system called LOCAL (Location and Context-Aware Learning). LOCAL uses location information and context as an aid to teaching and learning process. A location system monitors the mobility of apprentices and, based on their physical position, explores educational opportunities (Barbosa et al., 2011).

LOCAL, whose architecture is shown in Figure 9, consists of seven components: (i) a system of user profiles, which stores data relevant to the process of teaching and learning, (ii) a location system, (iii) a Personal Assistant (PA) that follows the user on his mobile device, (iv) a repository of learning objects, (v) a system for sending contextualized messages, (vi) a task scheduling event system and (vii) the Tutor, an analysis engine that performs inference using data provided by the profile and location systems.

The gathering of LOCAL's profile system information is explicit. This means that users' (learners) relevant data need to be pre-populated in the system. Furthermore, the LOCAL profile system is static, which means that

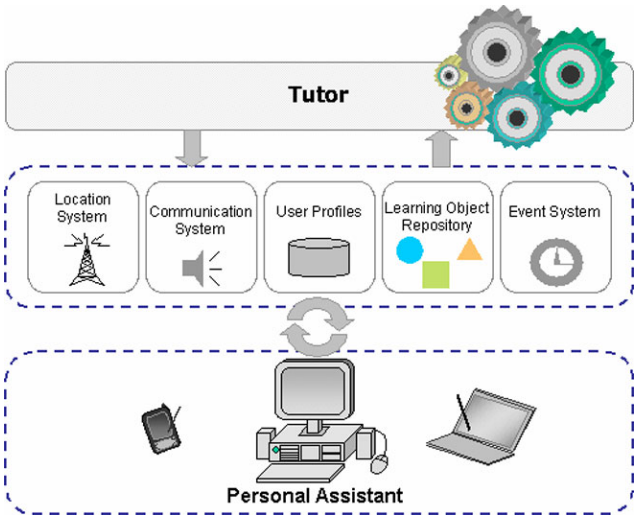

Fig. 9 LOCAL architecture (Barbosa et al., 2011)

data is not automatically updated in response to actions taken by learners or external events. Since eProfile is a dynamic entity model with implicit information collection, integrating these two models result in the enhancement of LOCAL capabilities.

LOCAL and eProfile integrate according to Figure 10. Besides LOCAL and eProfile, Figure 10 presents an application called trail manager. A trail manager is an external program that stores the trails, processes them and forwards them to eProfile, as described in Section **??**. For this scenario, we created a simple trail manager, which just receives, stores and forwards the trails to eProfile upon request.

Integration between eProfile and LOCAL was based on the following changes to LOCAL: (i) we included a connection interface with eProfile, able to generate learners entity models and rules for profile processing, (ii) we included a connection interface with the trail manager, to allow the location events and actions undertaken on the Personnel Assistant to be sent to the trail manager, and (iii) we included a communication interface with eProfile, where the LOCAL User Profiles module is fed by profiles generated by eProfile.

The first step of integration occurs when the LOCAL communicates with eProfile for the first time. In that moment, LOCAL sends to eProfile the entity model and the rules for the profile generation. This information set is registered in eProfile through the Configurator Agent (arrow "a"), which organizes this information and registers it in the Model and Rules databases.

As learners use LOCAL, their location changes are identified by the Location System module, and their actions are done on the Personal Assistant. These modules record events in the trail manager (arrows "b" and "c"), which in turn sends them to the Converter eProfile agent (arrow "d"). Trails sent are stored in the trail database. These trails are periodically processed according

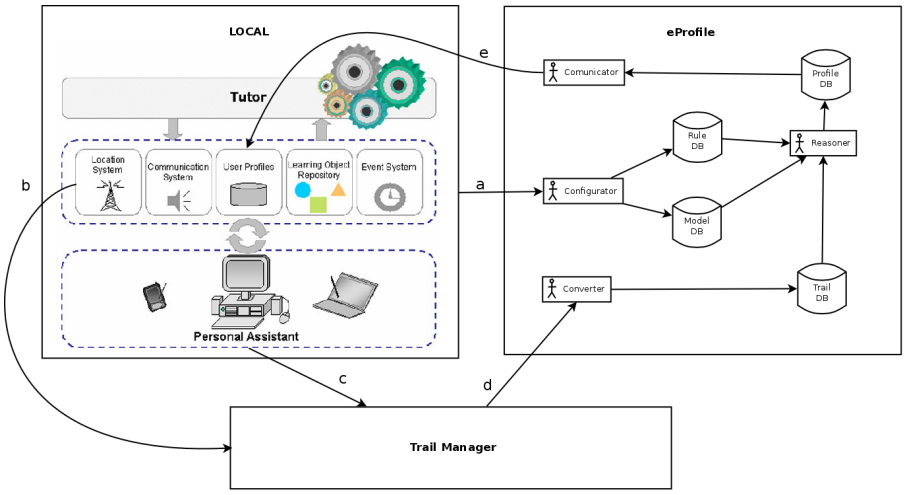

Fig. 10 eProfile/LOCAL Integration

to the models and rules defined initially, and a inference process based on this information generates profiles, which are sent to the LOCAL User Profiles module (arrow "e") by the communicator agent. These profiles help LOCAL know its users (learners) in order to put them in contact and distribute content in the most contextualized possible way.

4.4 eProfile/UniManager Integration

UniManager is a administrative software used by the university for registration and management of students, professors and disciplines. This application model is shown in Figure 11.

UniManager has two interfaces: administrative and user. The administrative interface is used by the management of the university to enroll subjects, professors and students. It is also used by students to carry out their registration for the next semester. This interface connects to a discipline manager, which manages the enrollment and prerequisites between disciplines.

The user interface is used in the professors' and students' mobile devices, and is connected to the university location system. Thus, all events recorded in UniManager from this interface contains the location of the student within the university, that is, the campus' region where the student is. The location system integrated to the user interface allows the system to check whether the student or the professor are present in class. The student performs the tests and hands out papers using this interface, and the professor evaluates the student tests and papers.

UniManager also contains a profile database. On this database are stored: (i) registration information about the disciplines, (ii) presences and absences of

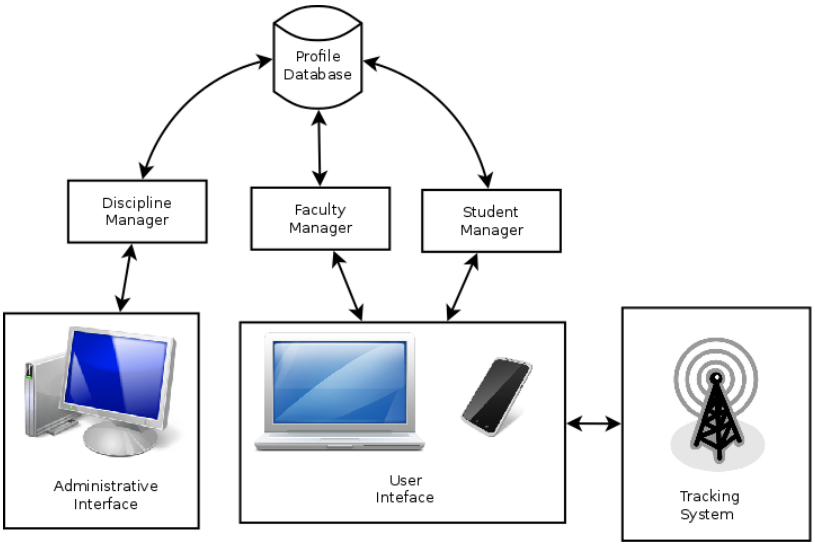

Fig. 11 UniManager Model

professors and students, and (iii) tests and papers handed out by the students, and (iv) students evaluations about the professor.

The profile database is dynamically updated based on recorded events, but is limited to data recorded in UniManager itself. Integration between eProfile and UniManager leverages the generation of more advanced profiles, using the information inferred by eProfile from the data generated by LOCAL. Using this data, it is possible to register the interests of the student in UniManager's profile database, in order to suggest him optional subjects that might be of his interest. Furthermore, the use of trails allows the profiles to be generated based on historical information, obtaining profile information that can be useful in administrative analysis (eg, what combination of professor and discipline historically resulted in higher grades for students in each discipline).

UniManager integrates to eProfile according to Figure 12. In the first communication between UniManager and eProfile, UniManager sends messages indicating to eProfile which are the entity models and the rules used to generate the profiles (arrow "a"). This information is received by the configurator agent, responsible for validating and storing the rules and models in their respective databases. As users make use of the UniManager interfaces, data is processed and stored by the Discipline, Professor and Student Manager modules in the trail manager, along with their location (arrows "b", "c" and "d"). The trail manager used by UniManager is the same one used by LOCAL. The trail manager periodically sends the trails generated by UniManager to the converter agent in eProfile (arrow "e"). This agent converts the trails to the eProfile format and stores them in the trail database. Once in possession of the trails, models and rules, the reasoner agent processes the information and

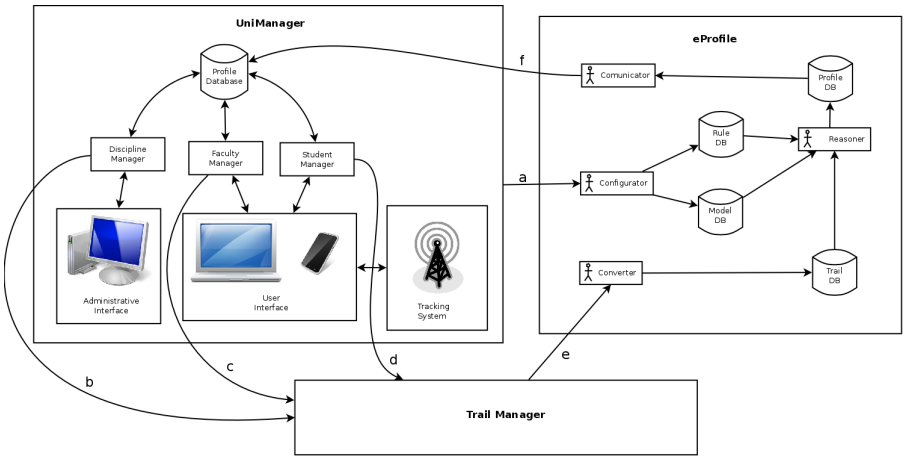

Fig. 12 UniManager/eProfile Integration

generates profiles that are sent to UniManager by the communicator agent (arrow "f").

4.5 Trails, Rules and Profiles

According to what was described in Sections 4.3 and 4.4, LOCAL and UniManager suffered three significant changes: (i) there were included communication interfaces with eProfile, so that applications can provide the entity models and rules, (ii) there were included communication interfaces with the trail manager, so that applications could register the events and (iii) the profile modules of both applications were modified to request profile updates from eProfile.

The scenario consists of three types of entities: (i) students, (ii) professors and (iii) disciplines. The "professor" and "student" entities represent, respectively, each one of the professors and students involved in the course. The "discipline" entity refers to each discipline of the course. A discipline could be represented either as an entity or as a single resource; in this scenario, we decided that it would be represented as an entity, since this allows it to create a trail, from which a profile can be obtained. This way, the profile of the discipline can contain fields that are updated automatically such as, for example, the average grade of the discipline.

According to what was described in Section 2.2, context-aware applications are interested in knowing **who**, **where**, **when** and **what** (ie, what the user is doing), so they can understand why the situation is occurring. The eProfile trail ontology, shown in Figure 4, defines that these questions are answered, respectively, by the classes *Entity*, *Location*, *Time* and *Activity*. Furthermore, the class *Participants* describes the other entities involved in this event. When a trail is stored in eProfile, information regarding the event should be inherited

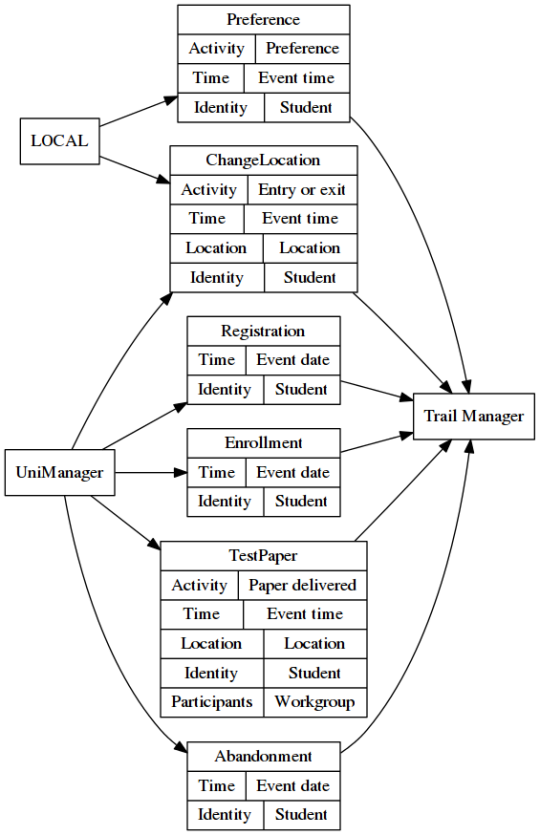

Fig. 13 Student trail ontology

from these classes, using as many classes as necessary so that the information in complete.

The students' trail, as registered by LOCAL and UniManager, contains the following kind of events: registration, discipline enrollment, entry and exit from locations inside the campus, handing out of test and papers, new preferences and abandonment of the course. The trail format is detailed in Figure 13. The following events are registered in the professor trail: registration, entry and exit from locations inside the campus and evaluation of students' tests and papers, according to the diagram shown in Figure 14. The discipline trail receives the following events: registration, beginning and end of semester and work group, according to Figure 15.

The ultimate goal of trail inference is the profile generation. For eProfile to be successful in generating profiles from trails, it requires the application to inform the entity models and the inference rules.

The entities models are the data structure used to store the profile of a entity. The entity model used by the LOCAL User Profile module is shown in Figure 16. The learners' profiles have two types of data: (i) persistent,

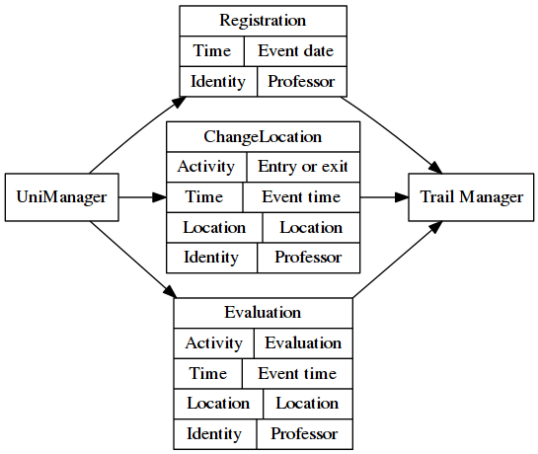

Fig. 14 Professor trail ontology

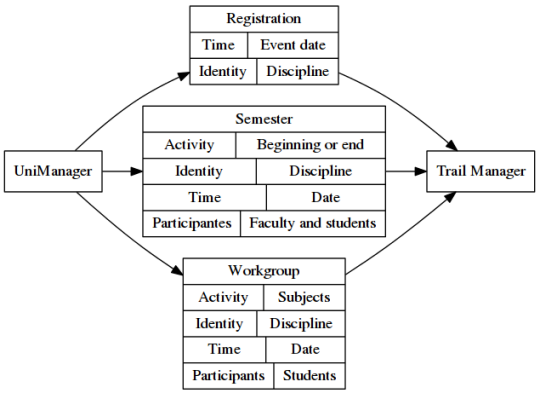

Fig. 15 Discipline trail ontology

associated to the learner in any situation, and (ii) contextual, associated with which the contexts in which the learner interacts.

Sections of the profile persistent block are (i) contact, which stores basic information of the user (such as name, phone number, among others), (ii) preferences, which stores the media type preferred by the user, and (iii) interests, which identifies the student's topics of interest. The interests section is categorized according to ACM's computational classification system (ACM, 2012). Sections of the contextual block are (i) relations, identifying relationships with other users within the same context, (ii) performance, which refers to the goals to be achieved by students and (iii) security, which stores credentials necessary to access different levels in specific contexts.

UniManager stores profiles of disciplines, professors and students. Its entity model is illustrated in Figure 17. The basic information, disciplines enrolled and notes/absences from the student profile, along with the disciplines infor-

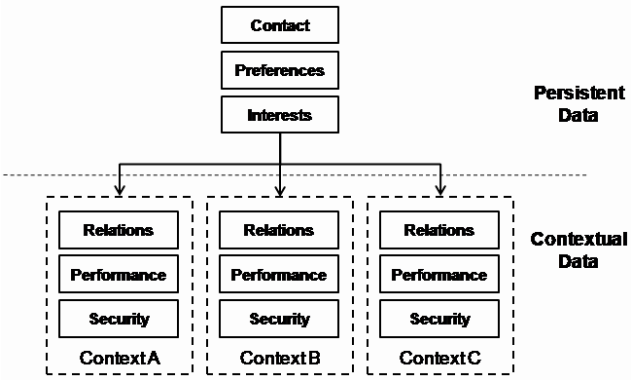

Fig. 16 LOCAL profile model (Barbosa et al., 2011)

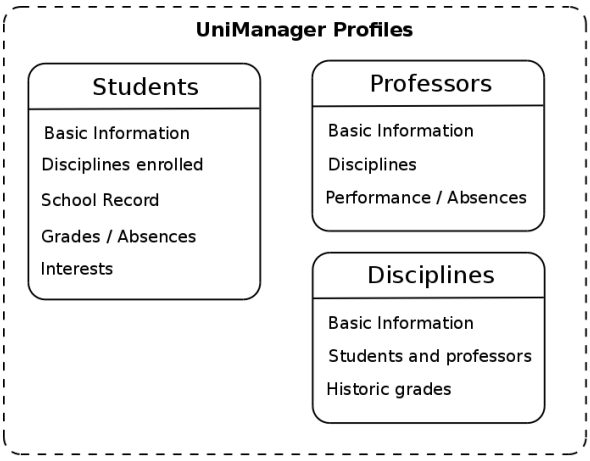

Fig. 17 UniManager profile model

mation, are used for managerial activities of the university, such as checking if the student can attend a particular discipline. Performance information about the professor and historical average grades of disciplines are used to adjust the professor with the discipline he has higher affinity or performance. Information about students interests are used to suggest optionals disciplines to them.

Each application define a set of rules for profile generation based on the trails registered by each one. An application has the option of using the trail of another application for processing rules and, consequently, profile generation, thus allowing the entity profile to be enriched by the surrounding environment.

The rule definition is written in OWL by creating classes that can be processed by a reasoner. Table 2 shows an example of a classroom where the professor registers in LOCAL a set of subjects that'll be discussed in class (event 1). This event generates a trail that is sent from LOCAL to the trail manager and, subsequently, to eProfile.

```
1  <EquivalentClasses>
2    <Class IRI="RelatedSubjects"/>
3    <ObjectIntersectionOf>
4      <Class IRI="Subject"/>
5      <ObjectSomeValuesFrom>
6        <ObjectProperty IRI="isRelated"/>
7        <Class IRI="?subject"/>
8      </ObjectSomeValuesFrom>
9    </ObjectIntersectionOf>
10 </EquivalentClasses>
11
12 <EquivalentClasses>
13   <Class IRI="StudentsSelected"/>
14   <ObjectIntersectionOf>
15     <Class IRI="Student"/>
16     <ObjectSomeValuesFrom>
17       <ObjectProperty IRI="hasInterest"/>
18       <Class IRI="RelatedSubjects"/>
19     </ObjectSomeValuesFrom>
20   </ObjectIntersectionOf>
21 </EquivalentClasses>
```

Fig. 18 OWL rule for identification of students interests

LOCAL then tries to identify students who have the same interests as the ones in the proposed debate, or interests related to these (event 7), by sending the OWL rules described in Figure 18. The first rule is called *RelatedSubjects* (line 2), where the subjects (class *Subject*, in line 4) related (line 6) to what will be discussed in class (line 7) are inferred. The expression `?Subject` is an eProfile extension to OWL, where this expression is replaced by the subjects that will be discussed, allowing the same rule to be used in different situations. This rule, after the inference process, results in the listing of all subjects (classes of type *Subject*) that relate to the subject that will be discussed, putting these classes as subclasses of class *RelatedSubjects*.

Using this rule, it is possible to find out the students (line 15) whose interests intersect with the interests found in the *RelatedSubjects* class (line 18). These students, in this ontology, become subclasses of class *StudentsSelected* (line 13).

Event 7 in Table 2 describes that two students did not match either group because they had no interest related to the subjects of debate, and LOCAL asked them on what subject they would like to participate (events 8 and 9), thus generating a trail event for these two students. Once the trail is registered, the student's profile is reprocessed according to the rules defined by LOCAL.

The header of the rule that updates the student interest field is shown in Figure 19. This header defines that the student profile (line 1) *Interest* field (line 2) will be updated using the subclasses of class *StudentInterest*, whose OWL rule is shown in Figure 20. It is also defined that a new interest is added to the interests that already exist (line 4), that this rule is updated every time an event is registered in the trail in which a subject is chosen (line 5). The OWL rule of Figure 20 defines that the *StudentInterest* class consists of the

```
1  Profile:   Student
2  Field:     Interest
3  Rule:      StudentInterest
4  Update:    addition
5  Frequency: onTrailEvent(SubjectChosen)
6  Expires:   never
```

Fig. 19 Student profile rule header

```
 1  <EquivalentClasses>
 2    <Class IRI="StudentInterest"/>
 3    <ObjectIntersectionOf>
 4      <Class IRI="SubjectChosen"/>
 5      <ObjectSomeValuesFrom>
 6        <ObjectProperty IRI="subjectChosen"/>
 7        <Class IRI="Subject"/>
 8      </ObjectSomeValuesFrom>
 9      <ObjectSomeValuesFrom>
10        <ObjectProperty IRI="hasStudent"/>
11        <Class IRI="?student"/>
12      </ObjectSomeValuesFrom>
13    </ObjectIntersectionOf>
14  </EquivalentClasses>
```

Fig. 20 Student profile OWL rule

subjects chosen (line 6) by the students who are part of this context (line 10 and 11) and that have chosen a particular subject (lines 6 and 7).

After the execution of the rule shown in Figures 19 and 20, the rule in Figure 18 runs again, this time identifying students who made their choice (event 10 of Table 2).

4.6 Prototype and Simulation

To simulate the model, we developed two programs:

1. A simulator, the simlates students, professors and disciplines using Uni-Manager and LOCAL;
2. A trail manager, which stores the generated trails and makes them available to eProfile.

To simulate the scenario as faithfully as possible, each entity was created with a set of characteristics that distinguish them from one another, and causes them to have different performances and take different decisions when faced with a choice. Professors are generated with random values of experience and rigor. These characteristics directly influence the quality of student learning, the difficulty of the course and the likelihood of dropout students. Students are generated with varying levels of commitment, intelligence and personal tastes. These characteristics influence student performance, the probability of

withdrawal of disciplines and course, and the choices that the student will make.

The scenario simulates a group of 40 students who conduct a course in computer science at a university. The simulation time is open, and it will be executed until all students graduate or drop out of the course.

A simulation starts with the creation of the discipline, student and professor entities. The initial profiles of the disciplines are then generated, and professors are randomly assigned to the disciplines. A loop then start, and it will only end when all students have completed the course or dropped out. Each iteration of the loop is a semester course and contains the following steps:

1. Students choose the disciplines they will attend and enroll;
2. Each day of the course is simulated, where students attend class and perform tests or deliver papers;
3. The students final grades are calculated, and the students decide if they will continue on the course or drop out.

Each course consists of 14 classes. There are two types of events that can occur in a class: (i) a debate or group work, and (ii) an individual paper or test. A class can have one of these two events, both or neither. Table 2 exemplifies a class in which the two types of events occurred.

For this simulation, it was defined that in each discipline there were studied a number of different subjects (from 3 to 5, randomly), and that each subject was related to 1 or 2 other subjects. For example, in the "Programming Languages" discipline, the subject of "C Language" was studied, among other programming languages. Later, in the discipline of "Data Structures", the subject of "Pointers" was studied. Since "C Language" and "Pointers" are related concepts, the subjects were linked, and if a student chose to study "C Language" using the LOCAL application, then the subject of "Pointers" is recommended to him.

Optional disciplines are also linked to subjects chosen by the students. Debate and paper subjects chosen by the students using LOCAL are used to suggest optional disciplines to the user on UniManager.

Figure 21 shows the results of the simulation. On this simulation, a group of 50 students goes through a complete graduation course. The simulator registers the percentage of the debates and papers subjects that were be suggested by the system, based on previous student choices. In the first semester, the system was able to suggest only 9% of the time, but as the students made more choices, the system was able to suggest more subjects, until the level of recommendation passed 90% in the 13th semester onwards.

To test the capability of changing the rules during the execution of eProfile, a new test was made. On this test, the execution was the same as the previous test until the 8th semester. After this semester, the rule for generating the listing of subjects related to the subject that is being debated or studied (as demonstrated in Figure 18) was changed: in addition to consider subjects related to the subject in point, the subjects chosen by the 5 most similar students to each student were also used as recommendation. In this context,

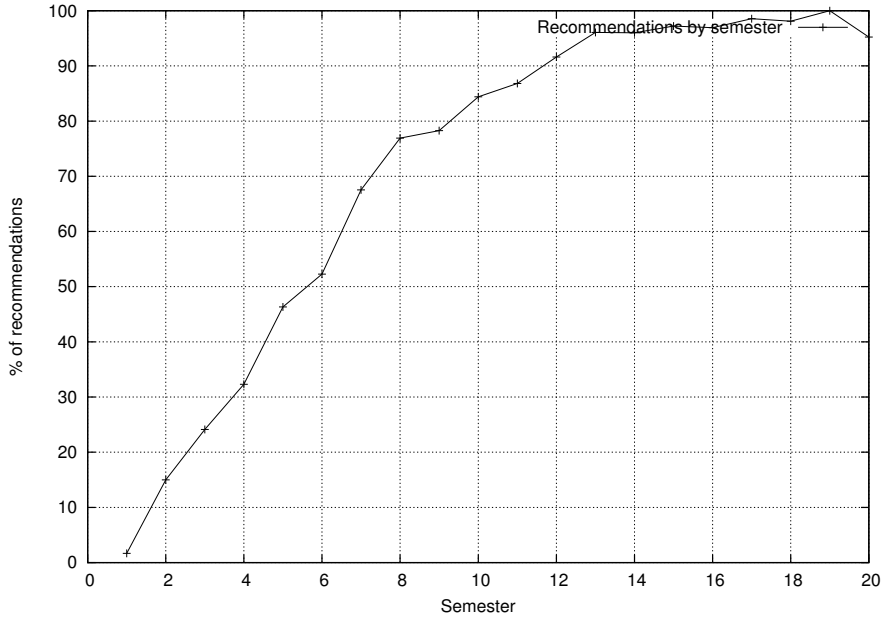

Fig. 21 Simulation results: automatic recommendations by semester

the most similiar student is the one who made more choices equals to another student.

Figure 22 shows the result of the second experiment. Comparing Figure 21 and Figure 22, it can be seen a sudden increase in the rate of subjects on the 8th semester, which is the result of the modification of the rules during the execution of the simulation.

5 Related Works

Currently, the areas of ubiquitous computing, context-awareness, trails and personalization are very promising, with a large number of papers being published annually on these subjects. However, we were unable to find a model that makes use of inference using trails in order to personalize the user experience within a generic domain. Thus, the model presented in this paper aims to fill this need.

Kim & Lee (2008) present a framework for profile management based on Web Services. The model provides a set of services to gather information necessary to create a profile: configuration, creation, exchange, processing and service. The model also applies dynamic configuration techniques for efficient profile management.

The model presented by Chellouche et al. (2010), like eProfile, presents a unified profile which can be used for various applications. However, differently

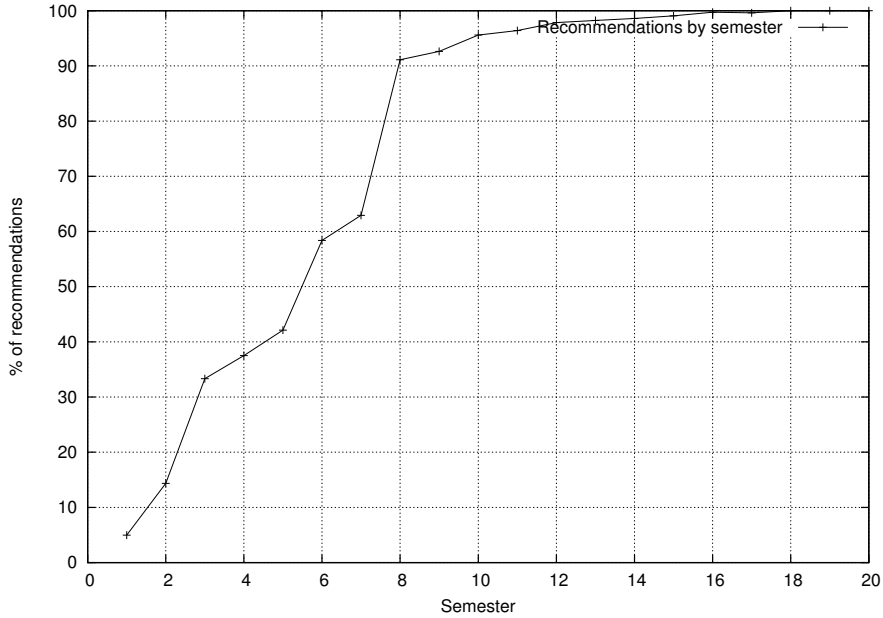

Fig. 22 Simulation results with rule modification during execution

from eProfile, the model assumes syntactic interoperability, which means that all applications need to record the events using the same format, which can cause the application to not be able register the event with the richness of detail required . Moreover, the model presented by Chellouche et al. (2010) is limited to a specific domain (multimedia services).

The UUCM model (Niederée et al., 2004), unlike other models, works in a generic domain. It does this by creating a basic ontology that can be extended by applications. Unlike other proposals, UUCM stores the profiles on the client rather than the server. This brings advantages and disadvantages. Among the advantages there can be mentioned the privacy and disconnected access. However, this approach limits the interoperability between systems, since each client gets a version of the client's profile, restricting the use of a unified profile.

PeLeP (Levis et al., 2008) supports automatic improvement of a learner's profile in education ubiquitous environments, using the history of apprentices to update the profiles through inference. eProfile uses a similar strategy. However, since PeLeP's inference is made on a single specific domain (education), in the eProfile model the inference rules are defined by the application and not the model, thus allowing eProfile to be applied to a generic domain. Moreover, eProfile's rules are dynamic (an application can define or remove rules dynamically), rather than being pre-defined in the model. This allows the evolution and refining of rules of inference.

The model presented by Razmerita (2011) aims to link tradition KM (Knowledge Management) with person KM. This is done through OntobUMf

Paper	Interope-rability	Dynamic profiles	Storage	Context-awareness	Uses trails	Domain
Kim & Lee (2008)	Sintatic	Yes	Server	No	No	Generic
Chellouche et al. (2010)	Sintatic	Yes	Server	Yes	No	Multimedia Services
Niederée et al. (2004)	Sintatic	Yes	Client	Yes	No	Generic
Levis et al. (2008)	No	Yes	Server	Yes	Partial	Education
Razmerita (2011)	Sintatic	Yes	Server	Yes	Partial	Generic
Tao et al. (2011)	Semantic	Yes	Server	Yes	Partial	Generic
eProfile	Semantic	Yes	Server	Yes	Yes	Generic

Table 3 Comparasion between presented papers

model, that manages user information. The architecture has three layers: front end, services and data. The model works in a generic domain and has three ontologies (user, domain and log), from which are fetched information that categorize the user in various ways, according to the level of activity, type of activity and level of knowledge sharing.

In the paper presented by Tao et al. (2011), it is proposed a ontology model for knowledge representation to gather personalized information. The model builds personalized user ontologies by extracting global information from LCSH system and by discovering information from local user repositories. The local information system is fed manually by the user. When information is needed, the processes both ontologies to generate the requested data.

Table 3 shows a comparison between eProfile and related works. The following factors are taken into consideration:

- Interoperability: considers if the model presents interoperability – if yes, whether it is sintatic or semantic – according to the definition in Section 1;
- Dynamic profiles: identifies if the model allows the profiles to be updated according to external events, without user interference;
- Storage: defines where the user profiles are stored;
- Context-awareness: considers if the user context is taken into consideration when forming the profile;
- Uses trails: defines if the user trails are used when building the profile. In the table, "Partial" means the model uses some type of historical information to form the profile, but it doesn't use the formal definition of trails;
- Domain: identifies if the profile is oriented to a specific domain, or if it allows different domains to be used.

eProfile was based on some elements of the works cited, such as automatic profile improvement, use of unified profiles and operation in generic domains. Comparing the related models, eProfile main contribution is the use of trails for extracting profiles, which facilitates the inference with context-sensitivity. Moreover, eProfile allows each application to register its trails in the format they find most helpful for them, allowing the richness of details registered by an application to be translated into a more accurate profile of the entity.

6 Conclusion

This article has proposed a model to support the management of profiles in ubiquitous environments, allowing entity trail inference through definition of rules by applications. Although eProfile was integrated with LOCAL and UniManager in the scenario, its proposal is generic enough to be applied to any other systems that use profiles from improvement of personalization. The use of automatic profiles allows ubiquitous systems to be more efficient in understanding user intent, because this means they can use more precise, up-to-date and broad information in a way that caters to the application specific need.

The objective of the scenario was to demonstrate that is viable to integrate ubiquitous systems with eProfile, enriching the profile generation of these applications and allowing them to adapt themselves to their users. The scenario demonstrate the four contributions of the model.

The first contribution is the use of trails for extracting profiles. The use of trails enrich the information database that can be used for generating profiles by providing a historical view of the entity actions within contexts. In the scenario, past decisions regarding debate and papers subjects are used to extract the preferences that will compose the students profiles.

The second contribution is the generation of dynamic profiles. By updating the profiles with new information present in the trails, it is possible to keep the profile in sync with everything that is happening with the entity. This allows the application to personalize the user experience. In the scenario presented, the profiles are updated every time a student makes a decision regarding a debate or paper subject.

The third contribution is the capability of managing dynamic inference rules and models. By allowing the rules and entity models to be updated at any time, eProfile facilitates the adaptation of the application to its user. A second experiment was made where the inference rules for profile generation were changed during the execution of the simulation, resulting in a positive change on the outcome of the scenario.

The fourth contribution is the semantic operability of the model. This opens the possibility that one application can use information generated by a different application to create the profile of its entities. This is demonstrated in the scenario where UniManager uses information generated by LOCAL (the optional disciplines chosen by the students) as part of the data that is used to create UniManager profiles. This information is later used to suggest optional disciplines to the students.

eProfile is a initial proposal, such as its integration with LOCAL and UniManager. The following activities will allow the continuity of the study: (i) more applications will be integrated to eProfile, which will test and expand eProfile capabilities, (ii) the performance of a great amount of trails needs to be evaluated, since the experiment considered a small number of interactions, (iii) the ideal evaluation would involve trails generated by real entities, instead of a simulation.

References

Abowd, Gregory D., Anind K. Dey, Peter J. Brown, Nigel Davies, Mark Smith, & Pete Steggles 1999. Towards a Better Understanding of Context and Context-Awareness. In Proceedings of the 1st international symposium on Handheld and Ubiquitous Computing, HUC '99, pages 304–307, London, UK. Springer-Verlag.

ACM 2012. ACM Computing Classification System. `http://www.acm.org/class/2012`.

Barbosa, Jorge Luis Victoria, Rodrigo Machado Hahn, Debora Nice Ferrari Barbosa, & Amarolinda Iara da Costa Zanel Saccol 2011. A ubiquitous learning model focused on learner interaction. International Journal of Learning Technology 2011, 6(1):62 – 83.

Carmagnola, Francesca, & Federica Cena 2009. User identification for cross-system personalisation. Inf. Sci., 179:16–32.

Chellouche, Soraya Ait, Julien Arnaud, & Daniel Négru 2010. Flexible user profile management for context-aware ubiquitous environments. In Proceedings of the 7th IEEE conference on Consumer communications and networking conference, CCNC'10, pages 980–984, Piscataway, NJ, USA. IEEE Press.

Daconta, Michael C., Leo J. Obrst, & Kevin T. Smith 2003. The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management. Wiley, Indianapolis.

Dey, A. 2001. Understanding and Using Context. Personal and Ubiquitous Computing, 6(1):4–7.

Dey, Anind K., Gregory D. Abowd, & Daniel Salber 2001. A conceptual framework and a toolkit for supporting the rapid prototyping of context-aware applications. Hum.-Comput. Interact., 16(2):97–166.

Edwards, W. Keith, Victoria Bellotti, Anind K. Dey, & Mark W. Newman 2003. The challenges of user-centered design and evaluation for infrastructure. In Proceedings of the SIGCHI conference on Human factors in computing systems, CHI '03, pages 297–304, New York, NY, USA. ACM.

Fielding, Roy Thomas 2000. Architectural styles and the design of network-based software architectures. PhD thesis. AAI9980887.

Heckmann, Dominik 2005. Ubiquitous User Modeling. PhD thesis, Department of Computer Science, Saarland University, Germany.

Hwaci 2012. SQLite Home Page. `http://www.sqlite.org/`. Accessed in 05/05/2012.

IEEE 1991. IEEE Standard Computer Dictionary. A Compilation of IEEE Standard Computer Glossaries. IEEE Std 610, page 1.

Kim, Kyung-Sik, & Jae-Dong Lee 2008. Profile Management Framework Based on Web Services for Providing Personalized Services. In Proceedings of the 2008 International Conference on Convergence and Hybrid Information Technology, ICHIT '08, pages 501–508, Washington, DC, USA. IEEE Computer Society.

Levis, Darci, Jorge L. V. Barbosa, Sérgio Crespo C. S. Pinto, & Debora Nice Ferrari Barbosa 2008. Aperfeiçoamento Automático do Perfil do Aprendiz em Ambientes de Educação Ubíqua. Revista Brasileira de Informática na Educação, 16(1).

Mulvenna, Maurice D., Sarabjot S. Anand, & Alex G. Büchner 2000. Personalization on the Net using Web mining: introduction. Commun. ACM, 43:122–125.

Niederée, Claudia, Avaré Stewart, Bhaskar Mehta, & M. Hemmje 2004. A Multi-Dimensional, Unified User Model for Cross-System Personalization. In E4PIA Workshop 2004.

Razmerita, Liana 2011. An Ontology-Based Framework for Modeling User Behavior - A Case Study in Knowledge Management. IEEE Transactions on Systems, Man, and Cybernetics, Part A, 41(4):772–783.

Rossum, Guido 1995. Python reference manual. Technical report, Amsterdam, The Netherlands, The Netherlands.

Satyanarayanan, M. 2001. Pervasive Computing: Vision and Challenges. IEEE Personal Communications, 8:10–17.

Silva, Jader, João Rosa, Jorge Barbosa, Débora Barbosa, & Luiz Palazzo 2010. Content distribution in trail-aware environments. Journal of the Brazilian Computer Society, 16:163–176. 10.1007/s13173-010-0015-1.

Silva, Jader, João Rosa, Jorge Barbosa, Débora N. F. Barbosa, & Luiz Antônio Moro Palazzo 2009. Content distribution in trail-aware environments. In Proceedings of the XV Brazilian Symposium on Multimedia and the Web, WebMedia '09, pages 15:1–15:8, New York, NY, USA. ACM.

Tao, Daniel, Yuefeng Li, & Ning Zhong 2011. A Personalized Ontology Model for Web Information Gathering. IEEE Transactions on Knowledge & Data Engineering, 23(4):496–511.

Tsarkov, D., & I. Horrocks 2006. FaCT++ Description Logic Reasoner: System Description. In Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006), volume 4130 of *Lecture Notes in Artificial Intelligence*, pages 292–297. Springer.

Viviani, Marco, Nadia Bennani, & Elod Egyed-Zsigmond 2010. A Survey on User Modeling in Multi-Application Environments. In The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10, pages 111–116. IEEE.

W3C 2011. OWL Web Ontology Language Guide. `http://www.w3.org/TR/owl-guide/`. Accessed in 05/05/2012.

Weiser, Mark 1991. The Computer for the 21st Century. Scientific America, pages 94–104.

Weiser, Mark, & John Seely Brown 1996. Designing Calm Technology. PowerGrid Journal.